

5 - 中央处理器

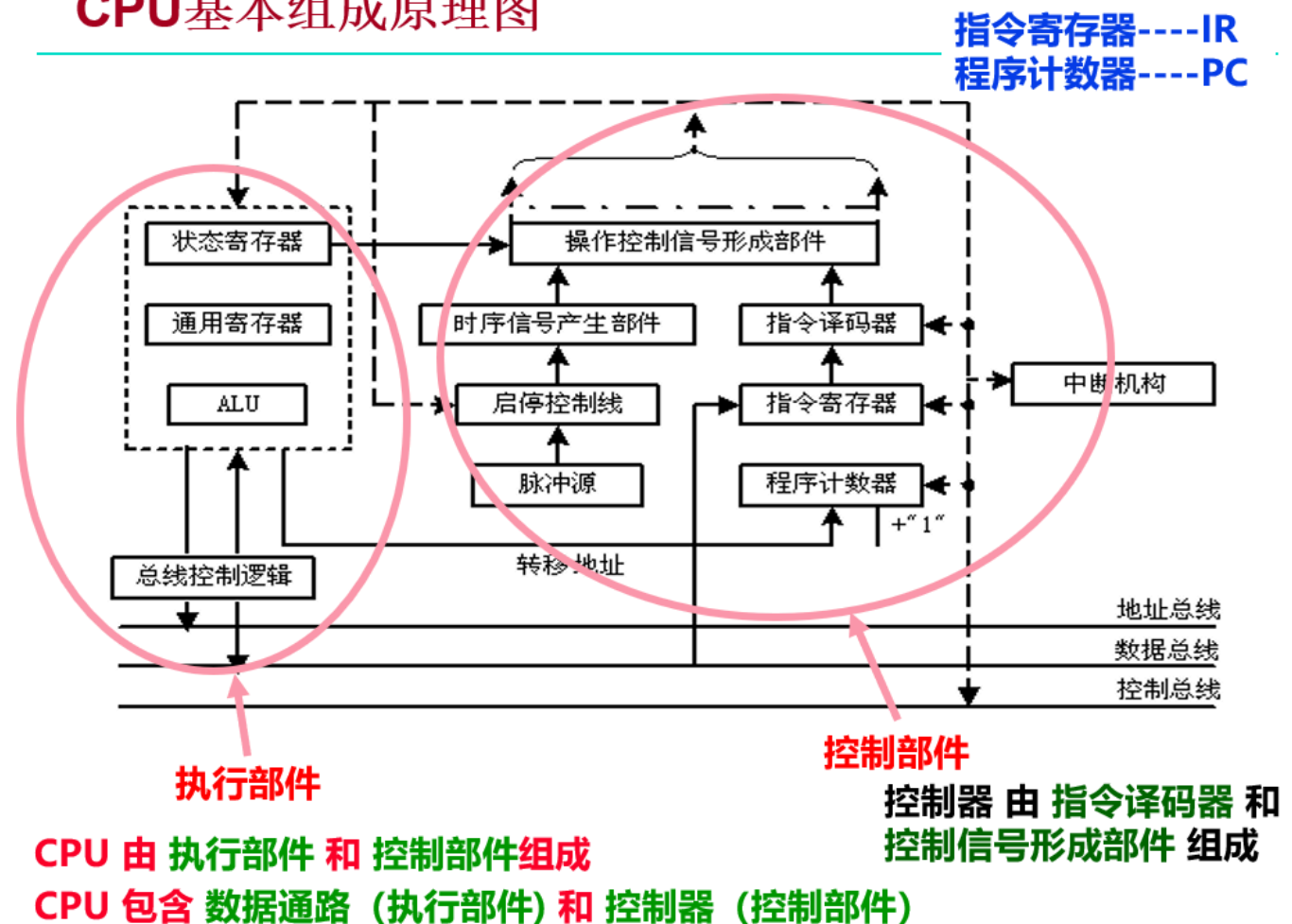
作者：Klein White

时间：2025-04-21

概要：

CPU 基本组成原理图

CPU基本组成原理图



数据通路（执行部件）：指令执行过程中，数据所经过的路径，包括路径中的部件。它是指令的执行部件

控制器：对指令进行译码，生成指令对应的控制信号，控制数据通路的动作。能对执行部件发出控制信号，是指令的控制部件

数据通路由两类元件组成：组合逻辑元件（操作元件，由组合逻辑电路实现）、时序逻辑元件（存储元件，分为寄存器和寄存器组，由时序逻辑电路实现）

元件之间通过 总线连接 / 分散连接 方式

数据通路的功能是进行数据的存储、处理和传送

数据通路是由操作元件和存储元件通过总线方式或分散方式连接而成的进行数据存储、处理、传送的路径

同步系统：所有动作有专门时序信号来定时，由时序信号规定何时发出什么动作

时序信号：同步系统中用于进行同步控制的定时信号

指令周期（机器周期）：取出并执行一条指令的时间。包括取指令、读操作数、执行并写结果等多个基本工作周期，称为机器周期

早期的时序系统由机器周期、节拍和脉冲构成，但现在已经不再使用

单总线数据通路 vs 多总线数据通路

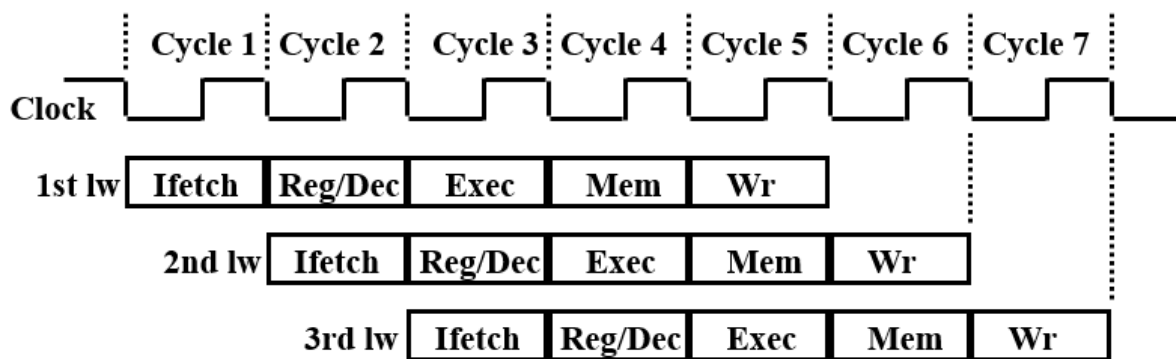
目前大都采用**流水线方式**执行指令，总线式的数据通路难以实现流水线执行

流水线处理器设计

Load 指令的五个阶段：

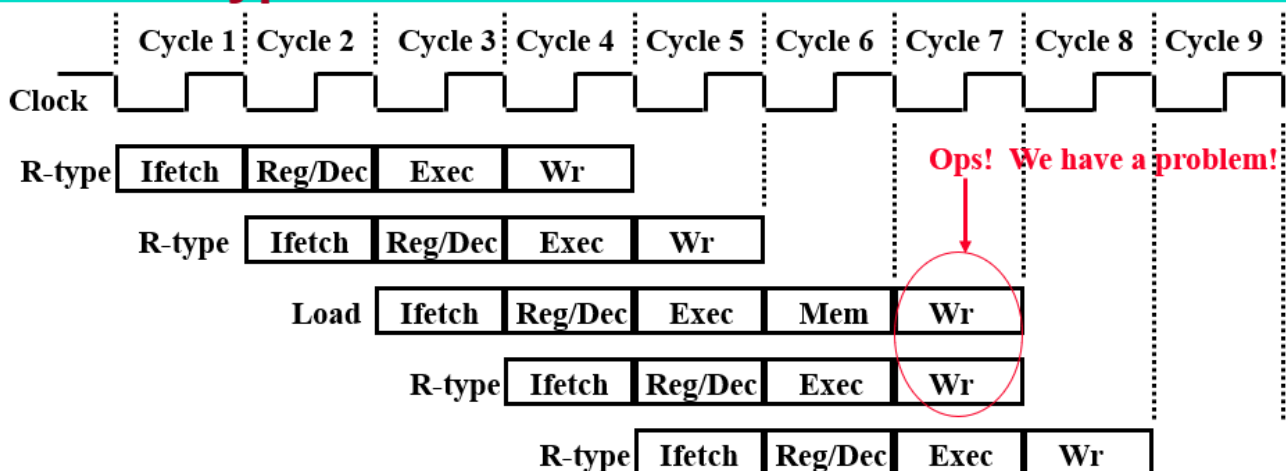
- Ifetch 取指：取指令并计算PC+4
- Reg/Dec 取数和译码：取数同时译码
- Exec 执行：计算内存单元地址
- Mem 读存储器：从数据存储器中读
- Wr 写存储器：将数据写到寄存器中

Load指令的流水线



- 每个周期有五个功能部件同时在工作
- 后面指令在前面完成取指后马上开始
- 每个load指令仍然需要五个周期完成
- 但是，吞吐率(throughput)提高许多，理想情况下，有：
 - 每个周期有一条指令进入流水线
 - 每个周期都有一条指令完成
 - 每条指令的有效周期(CPI)为1

R-type 指令的四个阶段：Ifetch 取指令、Reg/Dec 取数并译码、Exec 执行、Wr 写寄存器
如果 R-type 和 Load 指令一起运行，那么可能会出现冲突的情况：

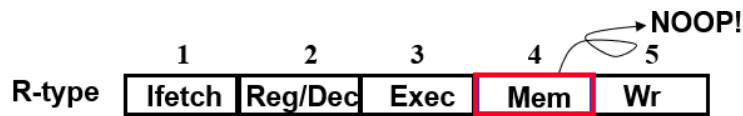


两条指令同时要写寄存器。把一个功能部件同时被多条指令使用的现象称为**结构冒险（或资源冲突）**

为了流水线能够正常工作，规定：

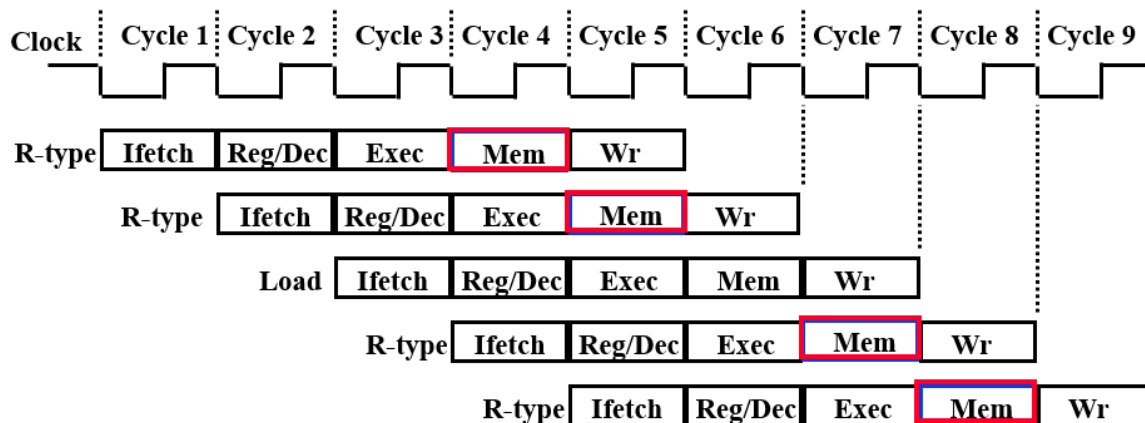
- 每个功能部件每条指令只能用一次，比如：wr 只能做一次

- 每个功能部件必须在相同的阶段被使用，像 R-type 指令，就应该在 Exec 和 Wr 之间插入一个 NOP 阶段以延迟写操作



◦ 加一个NOP阶段以延迟“写”操作:

- 把“写”操作安排在第5阶段, 这样使R-Type的Mem阶段为空NOP



这样使流水线中的每条指令都有相同多个阶段!

流水线工作方式下，虽然每条指令的执行时间不缩短，但是总体的执行时间大大提升，且提高了指令的吞吐率

流水线冒险处理

流水线的三种冒险 / 冲突情况:

1、**结构冒险 (资源冲突)**: 同一个部件同时被不同指令所使用

- 解决1: 一个部件每条指令只能用一次，且只能在特点周期使用
- 解决2: 设置多个部件
- 解决3: 将每个部件的读口和写口独立出来

2、**数据冒险 (数据依赖)**: 后面指令用到前面指令结果数据时，前面指令的结果还没产生

- 解决1: 采用转发技术
- 解决2: Load-Use 冒险需要一次阻塞
- 解决3: 编译程序优化指令的顺序

3、**控制冒险**: 转移或异常改变执行流程，后继指令在目标地址产生前已被取出

- 解决1: 采用静态或动态分支预测
- 解决2: 编译程序优化指令顺序

结构冒险解决

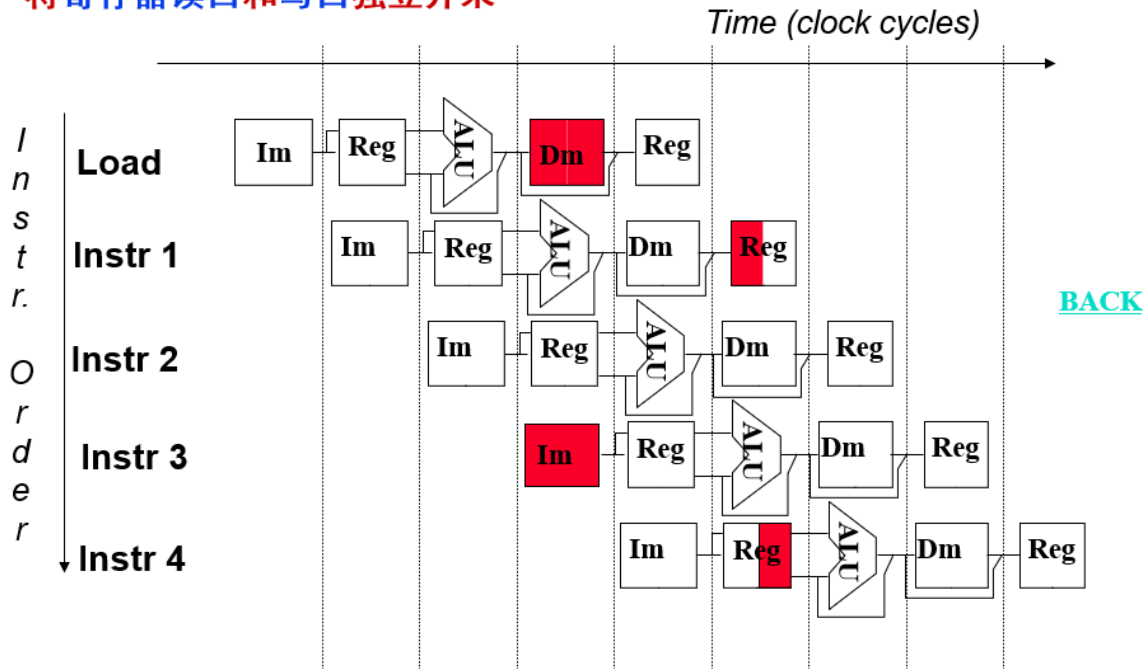
Structural Hazard的解决方法

为了避免结构冒险，规定流水线数据通路中功能部件的设置原则为：

每个部件在特定的阶段被用！（如：ALU总在第三阶段被用！）

将Instruction Memory (Im) 和 Data Memory (Dm)分开

将寄存器读口和写口独立开来



数据冒险

就类似于操作系统中进程间的同步，当要修改某个数据的时候，其他指令已经读取了该寄存器的数据，但是读到的是旧的数据，而不是新的数据

add r1, r2, r3

哪条是新的值？

sub r4, r1, r3

读r1时, add指令正在执行加法(EX段), 老值!

and r6, r1, r7

读r1时, add指令正在传递加法结果(M段), 老值!

or r8, r1, r9

读r1时, add指令正在写加法结果到r1(WB段), 老值!

xor r10, r1, r11 读r1时, add指令已经把加法结果写到r1, 新值

补充: 三类数据冒险现象

画出流水线图能很清楚理解!

RAW: 写后读(基本流水线中经常发生, 如上例)

WAR: 读后写(基本流水线中不会发生, 乱序执行时会发生)

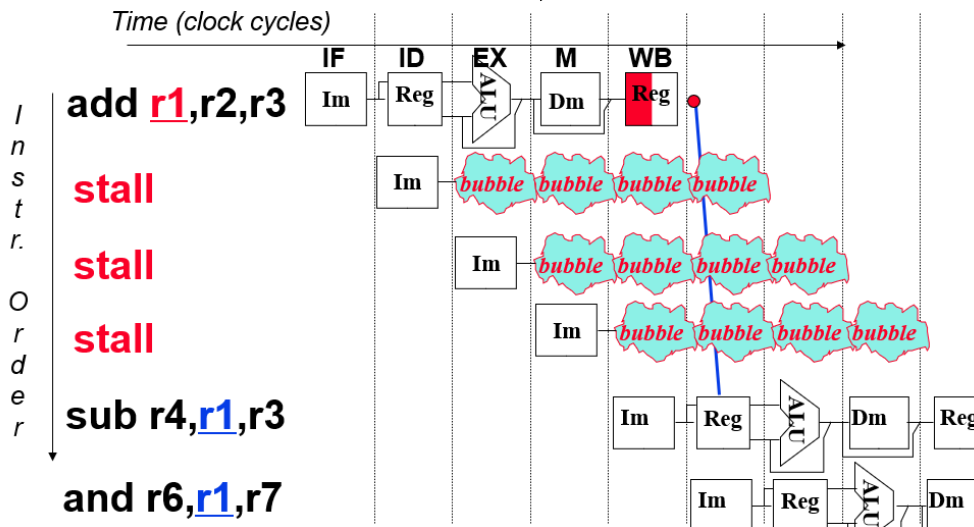
WAW: 写后写(基本流水线中不会发生, 乱序执行时会发生)

本讲介绍基本流水线, 所以仅考虑RAW冒险

解决方法:

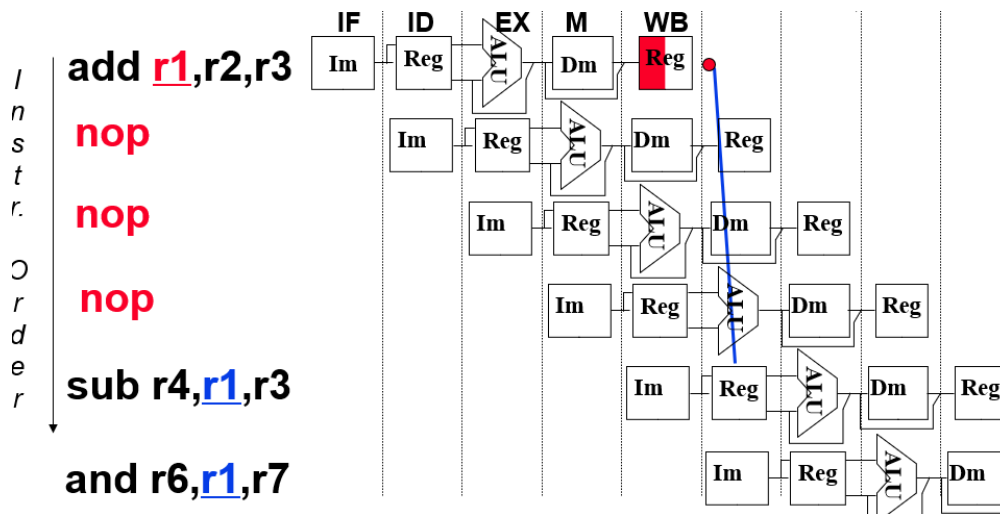
1、硬件阻塞: 硬件上通过阻塞(stall)方式阻止后续指令执行, 延迟到有新值以后
该做法称为流水线阻塞, 也叫插入气泡 Bubble

缺点: 控制比较复杂, 需要改数据通路; 指令被延迟三个时钟执行

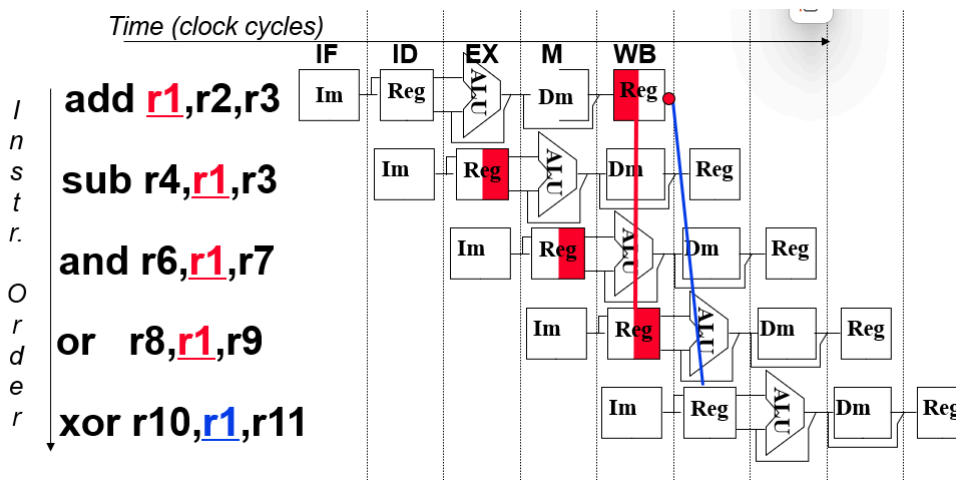


2、软件插入 NOP 指令: 由编译器插入三条NOP指令, 浪费三条指令的空间和时间, 是最差的做法。

优点: 数据通路简单, 无需修改数据通路

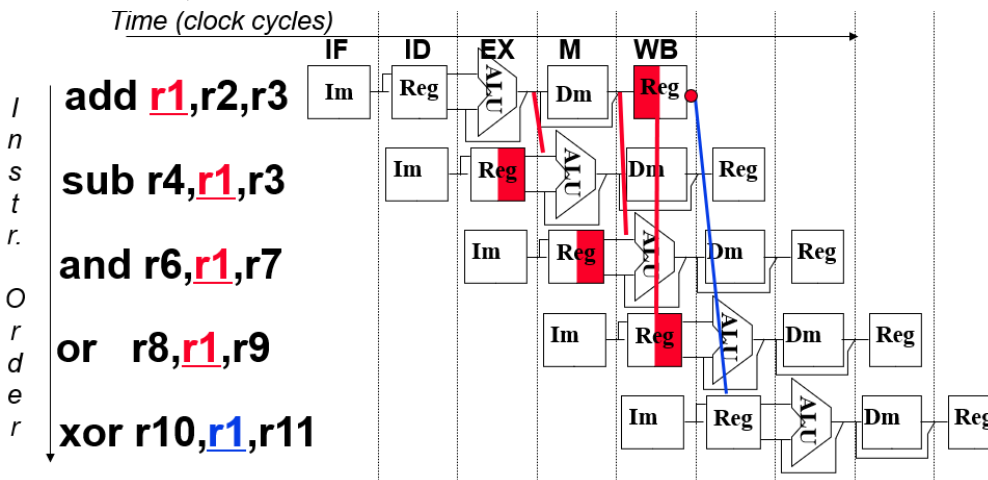


3、合理实现°寄存器堆的读/写操作，即同一周期内寄存器堆先写后读，但是这不能解决所有数据冒险



4、转发技术：若相关数据是 ALU 结果，那么可以通过转发解决，也就是把数据从流水段寄存器中直接取到ALU的输入端；

若相关数据是上条指令 DM 要写入的内容，那么不能转发解决，需要在随后指令阻塞一个时钟或加 NOP 指令，这样也叫做 Load-Use 数据冒险



5、编译优化

控制冒险解决方法

1、**硬件上阻塞**（stall）分支指令后的指令执行

2、**软件上插入指令**

3、分支预测：分为**静态预测和动态预测**

静态预测：总是预测条件不满足(not taken)，即：继续执行分支指令的后续指令

动态预测：根据程序执行的历史情况进行动态预测调整，能达90%的预测准确率

4、延迟分支：通过**编译优化指令顺序**